

Note di progetto — Benchmark Mandelbrot

Alberto

30 aprile 2026

Scopo

Questo documento serve come promemoria personale della fase di benchmark del progetto Mandelbrot in C++. L'obiettivo di questa fase non era ancora migliorare la qualità del render finale, ma capire dove il programma spende davvero tempo e quali direzioni di ottimizzazione hanno senso.

Contesto

La pipeline del progetto, in questa fase, è stata separata in:

1. costruzione del piano complesso;
2. controllo di appartenenza al set di Mandelbrot (`mandel_check`);
3. colorazione e scrittura finale.

Dai primi test è emerso subito che il vero collo di bottiglia non è il rendering finale, ma il `mandel_check`. Per questo motivo tutta la fase di analisi è stata concentrata quasi esclusivamente su questa parte.

Configurazione dei benchmark

Per i test è stato mantenuto fisso il viewport:

$$x \in [-0.8, -0.7], \quad y \in [0.05, 0.15]$$

Sono stati invece variati:

- la risoluzione della griglia (`nx`, `ny`);
- il numero massimo di iterazioni (`maxiter`).

Le risoluzioni testate sono state:

- 2000×2000
- 5000×5000
- 8000×8000
- 10000×10000

I valori di `maxiter` testati sono stati:

50, 100, 200, 400, 800

Tabella 1: Risultati dei test (Set 2). I tempi mostrano una correlazione lineare tra il numero di iterazioni e il tempo di calcolo di Mandelbrot.

Grid ($nx \times ny$)	Max Iter	Plane (ms)	Mandelbrot (ms)	Total (ms)
2000×2000	50	98	918	1016
	100	98	1730	1829
	200	98	3287	3386
	400	98	6307	6406
	800	100	12230	12330
5000×5000	50	610	5731	6341
	100	613	10805	11419
	200	611	20539	21151
	400	609	39394	40004
	800	610	76424	77035
8000×8000	50	1551	14666	16218
	100	1550	27690	29240
	200	1555	52585	54141
	400	1554	100851	102406
	800	1557	195679	197236
10000×10000	50	2433	23101	25535
	100	2418	43204	45623
	200	2428	82143	84572
	400	2445	157541	159986
	800	2446	305641	308088

Dati raccolti

Cose osservate

1. Il collo di bottiglia è chiaramente `mandel_check`

Il tempo speso nel controllo di appartenenza domina sempre il tempo totale. La costruzione del piano pesa relativamente poco, mentre la colorazione e il render finale sono ancora meno rilevanti.

Conclusione pratica: in questa fase del progetto ha senso ignorare quasi del tutto il render e concentrare tutte le energie su `mandel_check`.

2. Il costo scala quasi linearmente con il numero di pixel

A `maxiter` fissato, quando aumenta la risoluzione aumenta quasi proporzionalmente anche il tempo del `mandel_check`.

Per esempio, a `maxiter` = 50:

- 2000×2000 : 918 ms
- 5000×5000 : 5731 ms
- 8000×8000 : 14666 ms
- 10000×10000 : 23101 ms

Questo significa che, almeno nella finestra analizzata, il kernel si comporta in modo regolare rispetto al numero di punti campionati.

3. Il costo cresce quasi linearmente anche con `maxiter`

A risoluzione fissata, raddoppiare `maxiter` fa quasi raddoppiare il tempo, ma leggermente meno del doppio.

Per esempio, a 5000×5000 :

- $50 \rightarrow 100$: $5731 \rightarrow 10805$
- $100 \rightarrow 200$: $10805 \rightarrow 20539$
- $200 \rightarrow 400$: $20539 \rightarrow 39394$
- $400 \rightarrow 800$: $39394 \rightarrow 76424$

Interpretazione: l'aumento di `maxiter` pesa molto, ma non tutti i punti sfruttano davvero tutto il budget di iterazioni, perché molti punti esterni divergono prima.

4. Il kernel è abbastanza prevedibile

Una stima media utile emersa dai benchmark è che il costo si comporta in prima approssimazione come:

$$T \approx C \cdot nx \cdot ny \cdot maxiter$$

con una costante empirica dell'ordine di pochi nanosecondi per pixel per iterazione.

Questo è importante perché permette di prevedere abbastanza bene il costo di un benchmark futuro prima ancora di lanciarlo.

Decisioni progettuali maturate

Da questa fase di misurazione sono emerse alcune decisioni abbastanza chiare.

1. Il render non è la priorità.
2. La priorità assoluta è ottimizzare `mandel_check`.
3. Prima vanno tentate ottimizzazioni algoritmiche.
4. Solo dopo avrà senso parallelizzare.

Direzioni di ottimizzazione considerate

Le idee emerse fin qui sono:

- compilazione in modalità `Release`;
- miglioramento del loop caldo del test punto per punto;
- riconoscimento più veloce dei punti interni;
- periodicity checking;
- sfruttamento della simmetria quando il viewport lo permette;
- in futuro, propagazione di informazione tra punti vicini o segmenti.

Una possibile idea personale da esplorare è la seguente: campionare in modo più rado lungo una riga o un segmento, riconoscere zone chiaramente uniformi, e concentrare il lavoro pieno solo nelle zone di bordo o ambigue. Questa idea per ora è ancora concettuale, ma è una direzione che vale la pena tenere in mente.

Cosa ricordare

Le cose più importanti da non dimenticare sono:

- il programma è già stato misurato in modo sistematico;
- i risultati sono coerenti e non casuali;
- il collo di bottiglia è stato identificato con chiarezza;
- il prossimo passo sensato non è cambiare tutto, ma migliorare `mandel_check` con criterio.

Prossimo passo concreto

Il prossimo passo da fare dovrebbe essere:

1. attivare la compilazione **Release**;
2. rifare almeno un benchmark di controllo;
3. poi intervenire sul test del punto interno;
4. solo dopo passare a periodicità e strategie più strutturate.

Tabella 2: Risultati dei test in *release mode*. Parametri costanti: $x \in [-0.8, -0.7]$, $y \in [0.05, 0.15]$.

Grid ($N \times N$)	Max Iter	Plane (ms)	Mandelbrot (ms)	Total (ms)
2000 $\times$ 2000	50	41	331	373
	100	42	641	684
	200	41	1229	1270
	400	42	2363	2406
	800	41	4618	4659
5000 $\times$ 5000	50	255	2089	2344
	100	256	4001	4257
	200	255	7679	7934
	400	257	14772	15029
	800	253	28943	29197
8000 $\times$ 8000	50	654	5384	6039
	100	660	10422	11082
	200	656	20106	20762
	400	655	38059	38715
	800	658	74095	74754
10000 $\times$ 10000	50	1023	8290	9313
	100	1021	15910	16931
	200	1021	30914	31936
	400	1027	59728	60755
	800	1022	116838	117860

Tabella 3: Confronto Prestazioni: Release vs. Debug/Standard. Lo Speedup indica il rapporto tra i tempi totali.

Grid ($N \times N$)	Max Iter	Total Release (ms)	Total Standard (ms)	Speedup
2000×2000	50	373	1016	2.72x
	800	4659	12330	2.65x
5000×5000	50	2344	6341	2.70x
	800	29197	77035	2.64x
8000×8000	50	6039	16218	2.68x
	800	74754	197236	2.64x
10000×10000	50	9313	25535	2.74x
	800	117860	308088	2.61x

Tabella 4: Risultati in **Release mode con funzioni inline** nel loop *mandel_set*. Parametri: $x \in [-0.8, -0.7]$, $y \in [0.05, 0.15]$.

Grid ($N \times N$)	Max Iter	Plane (ms)	Mandelbrot (ms)	Total (ms)
2000×2000	50	45	336	381
	100	44	649	694
	200	43	1246	1290
	400	43	2401	2445
	800	44	4692	4736
5000×5000	50	261	2104	2365
	100	268	4052	4321
	200	265	7803	8068
	400	262	15020	15283
	800	266	29967	30233
8000×8000	50	655	5395	6051
	100	661	10347	11009
	200	660	19998	20658
	400	662	38868	39530
	800	662	76883	77546
10000×10000	50	1029	8472	9501
	100	1032	16519	17551
	200	1033	33622	34656
	400	1034	60015	61049
	800	1034	117234	118268

Tabella 5: Risultati in **Release mode**: funzioni inline e **check del cardioide**. Parametri costanti: $x \in [-0.8, -0.7]$, $y \in [0.05, 0.15]$.

Grid ($N \times N$)	Max Iter	Plane (ms)	Mandelbrot (ms)	Total (ms)
2000×2000	50	44	189	234
	100	43	336	380
	200	43	600	643
	400	43	1101	1144
	800	42	2052	2095
5000×5000	50	261	1168	1430
	100	265	2099	2364
	200	264	3757	4021
	400	266	6871	7138
	800	266	12858	13125
8000×8000	50	668	2983	3651
	100	664	5353	6017
	200	663	9692	10356
	400	665	17598	18263
	800	661	34317	34978
10000×10000	50	1038	5171	6209
	100	1036	8656	9693
	200	1034	15274	16308
	400	1038	29963	31001
	800	1034	51428	52463

Tabella 6: Risultati in **Release mode**: implementazione **check cardioide + bulbo periodo 2** (inline). Parametri: $x \in [-0.8, -0.7]$, $y \in [0.05, 0.15]$.

Grid ($N \times N$)	Max Iter	Plane (ms)	Mandelbrot (ms)	Total (ms)
2000×2000	50	48	89	137
	100	44	130	174
	200	43	182	226
	400	43	255	299
	800	44	357	402
5000×5000	50	269	550	819
	100	275	807	1083
	200	271	1146	1418
	400	272	1591	1863
	800	269	2220	2489
8000×8000	50	679	1413	2092
	100	673	2061	2735
	200	674	2928	3602
	400	671	4074	4746
	800	668	5656	6324
10000×10000	50	1034	2224	3259
	100	1038	3214	4253
	200	1035	4551	5586
	400	1044	6361	7406
	800	1040	8843	9883

1 Implementazione del Test Benchmarking (main.cpp)

Il seguente codice implementa un sistema di test automatizzato che cicla attraverso diverse risoluzioni e numeri di iterazioni, producendo un output in formato CSV per l'analisi delle prestazioni.

Listing 1: Main per il benchmarking automatizzato

```
1  #include <chrono>
2  #include <exception>
3  #include <iostream>
4  #include <vector>
5
6  #include "fractal_pl.hpp"
7  #include "mandel_set.hpp"
8
9  using clock_type = std::chrono::steady_clock;
10
11 int main() {
12     try {
13         const double xmin = -0.80;
14         const double xmax = -0.70;
15         const double ymin = 0.05;
16         const double ymax = 0.15;
17
18         const std::vector<std::size_t> resolution_values{2000, 5000,
19             8000, 10000};
20         const std::vector<int> iteration_values{50, 100, 200, 400, 800};
21
22         // Header CSV per facilitare l'importazione dei dati
23         std::cout << "xmin,xmax,ymin,ymax,nx,ny,maxiter,plane_ms,
24             mandel_ms,total_ms\n";
25
26         for (std::size_t resolution : resolution_values) {
27             for (int maxiter : iteration_values) {
28                 auto total_start = clock_type::now();
29
30                 // 1. Misurazione tempo creazione griglia
31                 auto plane_start = clock_type::now();
32                 fractal_pl plane{xmin, xmax, ymin, ymax, resolution,
33                     resolution};
34                 auto plane_end = clock_type::now();
35
36                 // 2. Misurazione tempo calcolo Mandelbrot
37                 auto mandel_start = clock_type::now();
38                 mandel_set::mandel_check(plane, maxiter);
39                 auto mandel_end = clock_type::now();
40
41                 auto total_end = clock_type::now();
42
43                 // Conversione in millisecondi
44                 auto plane_ms = std::chrono::duration_cast<std::chrono::
45                     milliseconds>(
46                     plane_end - plane_start).count();
47
48                 auto mandel_ms = std::chrono::duration_cast<std::chrono::
49                     milliseconds>(
50                     mandel_end - mandel_start).count();
51
52                 std::cout << xmin << ", " << xmax << ", " << ymin << ", " << ymax << ", " << resolution << ", " << maxiter << ", " << plane_ms << ", " << mandel_ms << ", " << total_ms << "\n";
53             }
54         }
55     }
56 }
```



```

47         auto total_ms = std::chrono::duration_cast<std::chrono::
           milliseconds>(
48             total_end - total_start).count();
49
50         // Output formattato CSV
51         std::cout << xmin << ',' << xmax << ',' << ymin << ','
           << ymax << ','
52             << resolution << ',' << resolution << ',' <<
           maxiter << ','
53             << plane_ms << ',' << mandel_ms << ',' <<
           total_ms << '\n';
54     }
55 }
56 }
57 catch (const std::exception& e) {
58     std::cerr << "Errore: " << e.what() << '\n';
59     return 1;
60 }
61
62 return 0;
63 }

```
